

KẾ HOẠCH TỔNG THỂ PHÁT TRIỂN VS CODE EXTENSION

Claude Code & Antigravity Engine Parity via AWS Bedrock API

Tác giả: Senior Engineering Team (10+ YOE)

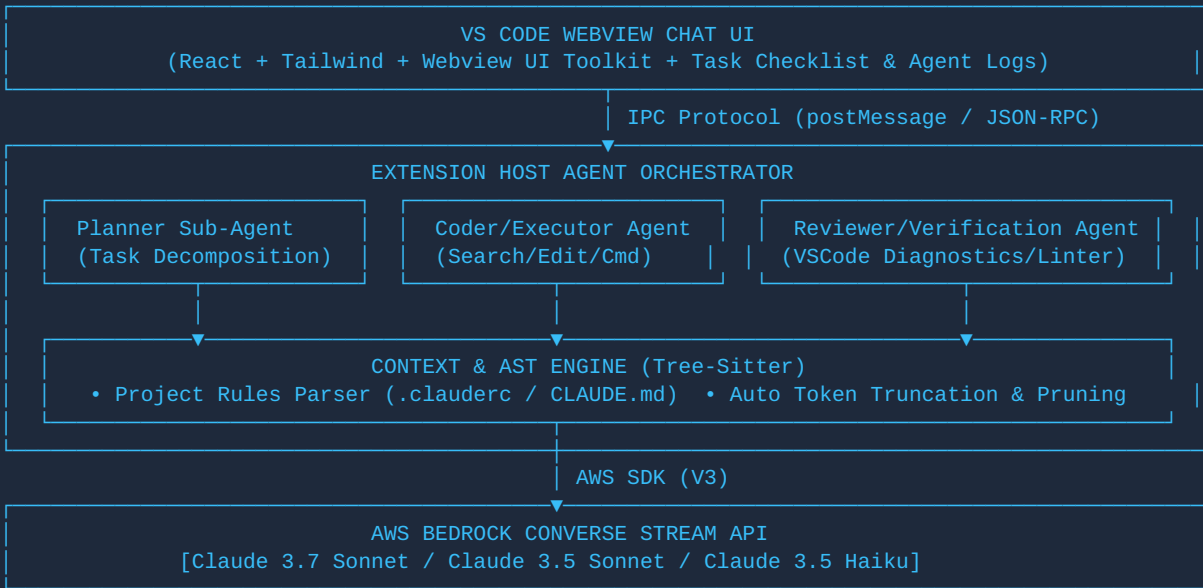
Môi trường target: VS Code / Antigravity Agent Studio

Backend Core: AWS Bedrock ConverseStream API

Thời gian: 12 Tuần (6 Phases)

1. TỔNG QUAN KIẾN TRÚC (ARCHITECTURAL BLUEPRINT)

Hệ thống được thiết kế theo mô hình **Multi-Agent Orchestrator** phân cấp, giả lập hoàn hảo các năng lực của **Claude Code CLI** và **Antigravity Engine**. Thay vì gửi toàn bộ codebase lên LLM gây lãng phí token và chậm trễ, hệ thống kết hợp **Tree-sitter AST Parsing**, **Vector RAG Search**, và vòng lặp **ReAct (Reasoning + Acting) execution loop** với kiểm soát an toàn nghiêm ngặt qua AWS Bedrock.



2. BẢNG ĐỐI CHIẾU NĂNG LỰC (100% PARITY MATRIX)

Tính năng Key	Claude Code / Antigravity Target	Giải pháp Kiến trúc Bedrock Extension	Phân loại
LLM Streaming & Tools	Hỗ trợ Claude 3.5/3.7 Streaming & Function Call	Sử dụng AWS Bedrock <code>ConverseStream</code> API kết hợp EventStream parser	CORE ENGINE
Sửa Code Tiết Kiệm Token	Search-and-Replace Block (Diff Patch)	Triển khai Tool <code>replace_string_in_file</code> thay vì ghi đè toàn bộ file	ADVANCED
Context & AST Parsing	Tree-sitter Code Graph & Definition Jump	Tích hợp WASM Tree-sitter để phân tích symbol graph, class, import dependency	ADVANCED
Project Rules Enforcement	Tự động tuân thủ <code>CLAUDE.md</code> & <code>.clauderc</code>	Parser tự động đọc các file config dự án inject trực tiếp vào Bedrock System Prompt	ADVANCED
Auto-Verification Loop	Tự bắt lỗi compile/linter ngay khi sửa xong	Hook vào <code>vscode.languages.getDiagnostics</code> để gửi lại lỗi gạch chân đỏ cho AI	ADVANCED
Multi-Agent Coordination	Planner Agent chia nhỏ task & Coder thi hành	Triển khai State Machine quản lý Todo List UI & định tuyến prompt cho sub-agents	ADVANCED
Safety & Sandboxing	Xác nhận lệnh nguy hiểm, lọc file nhạy cảm	Granular Approvals system + Lọc <code>.env</code> , <code>credentials</code> , khóa lệnh <code>rm -rf</code>	SECURITY

3. LỘ TRÌNH TRIỂN KHAI CHI TIẾT (12 TUẦN)

TUẦN 1 - 2 | PHASE 1

Bedrock ConverseStream Integration & ReAct Agent Loop

- Khởi tạo Extension Host boilerplate bằng TypeScript + ESBUILD.
- Tích hợp `@aws-sdk/client-bedrock-runtime` với đầy đủ auth (AWS Credentials, SSO Profile, IAM Roles).
- Xây dựng Vòng lặp Agentic ReAct Engine lắng nghe `toolUse` block từ stream.

TUẦN 3 - 4 | PHASE 2

Core Toolsets & VS Code Terminal Engine

- Triển khai bộ công cụ cơ bản: `read_file`, `list_dir`, `grep_search`.
- Xây dựng Tool `execute_terminal_command` sử dụng VS Code Terminal API, bắt stdout/stderr và exit code.
- Thiết lập màn hình xác nhận an toàn (User Approval Modal) trước khi chạy lệnh hệ thống.

TUẦN 5 - 6 | PHASE 3

Search & Replace Engine & Diff Visualizer UI

- Triển khai Tool `replace_string_in_file`: Tìm đoạn văn bản chính xác và thay thế (Tránh ghi đè cả file lớn).
- Sử dụng `vscode.diff` API mở tab so sánh code đề xuất trước khi Apply.
- Phát triển Webview Chat UI bằng React + Tailwind CSS, render Markdown, Code highlight và Typewriter streaming.

TUẦN 7 - 8 | PHASE 4

AST Tree-Sitter & Codebase Context Indexer

- Tích hợp WebAssembly Tree-sitter để parse AST cho JS/TS, Python, Go, Java.
- Tự động dựng bản đồ quan hệ hàm/class để hỗ trợ lệnh "Find Implementations/References".
- Tích hợp `Project Rules Reader` : Tự động phát hiện `CLAUDE.md` hoặc `.cursorrules` và gắn vào System Prompt.

TUẦN 9 - 10 | PHASE 5

Auto-Verification Loop (Diagnostics Integration)

- Tích hợp với `vscode.languages.onDidChangeDiagnostics`.
- Sau khi Agent sửa file, tự động chờ 1.5s để VS Code Linter/Compiler chạy, thu thập toàn bộ Error/Warning.
- Nếu có lỗi phát sinh, tự động gửi `toolResult` chứa danh sách lỗi để Agent sửa lại cho đến khi pass hẳn.

TUẦN 11 - 12 | PHASE 6

Planner Sub-Agent & Task Execution UI

- Triển khai Planner Agent: Tự động phân tích requirement phức tạp thành danh sách các bước (Todo List).
- Hiển thị Checklist động trên Webview Sidepanel, cập nhật trạng thái realtime (Pending -> In Progress -> Done).
- Tối ưu hóa Token Budgeting & Auto-pruning lịch sử chat khi chạm ngưỡng tối đa của Bedrock models.

4. THIẾT KẾ MÃ NGUỒN TRỌNG TÂM (TECHNICAL CODE BLUEPRINTS)

4.1. Bedrock Agentic Loop & Tool Execution (TypeScript)

```
import { BedrockRuntimeClient, ConverseStreamCommand } from "@aws-sdk/client-bedrock-runtime";

export async function runAgentLoop(userPrompt: string, history: any[], tools: any[]) {
  const client = new BedrockRuntimeClient({ region: process.env.AWS_REGION || "us-east-1" });

  let messages = [...history, { role: "user", content: [{ text: userPrompt }] }];
  let continueLoop = true;

  while (continueLoop) {
    const command = new ConverseStreamCommand({
      modelId: "us.anthropic.claude-3-7-sonnet-20250219-v1:0",
      messages: messages,
      toolConfig: { tools: tools }
    });

    const response = await client.send(command);
    let assistantContent: any[] = [];
    let toolRequests: any[] = [];

    for await (const chunk of response.stream) {
      if (chunk.contentBlockDelta?.delta?.text) {
        emitToUI("stream_text", chunk.contentBlockDelta.delta.text);
      }
      if (chunk.contentBlockStart?.start?.toolUse) {
        toolRequests.push(chunk.contentBlockStart.start.toolUse);
      }
    }

    if (toolRequests.length === 0) {
      continueLoop = false; // Thuật toán dừng khi LLM không yêu cầu thêm Tool
    } else {
      for (const toolReq of toolRequests) {
        const result = await executeToolHandler(toolReq.name, toolReq.input);
        messages.push({
          role: "user",
          content: [{ toolResult: { toolUseId: toolReq.toolUseId, content: [{ json:
result }] } } ] }
        });
      }
    }
  }
}
```

4.2. Tool Search & Replace Block (Tránh ghi đè file lớn)

```
export const replaceStringToolSchema = {
  toolSpec: {
    name: "replace_string_in_file",
    description: "Replace an exact block of code with new code in a specific file.",
    inputSchema: {
      json: {
        type: "object",
        properties: {
          path: { type: "string", description: "Relative file path" },
          old_string: { type: "string", description: "Exact code block to be replaced" },
          new_string: { type: "string", description: "New code block to insert" }
        },
        required: ["path", "old_string", "new_string"]
      }
    }
  }
};

export async function handleReplaceString(path: string, oldStr: string, newStr: string) {
  const uri = vscode.Uri.file(workspaceRoot + "/" + path);
  const document = await vscode.workspace.openTextDocument(uri);
  const content = document.getText();

  if (!content.includes(oldStr)) {
    return { success: false, error: "Target old_string not found in file. Ensure exact match including whitespace." };
  }

  const updatedContent = content.replace(oldStr, newStr);
  const edit = new vscode.WorkspaceEdit();
  edit.replace(uri, new vscode.Range(0, 0, document.lineCount, 0), updatedContent);
  await vscode.workspace.applyEdit(edit);
  return { success: true, message: "Successfully updated code block." };
}
```

4.3. Auto-Verification Loop (VS Code Diagnostics Hook)

```
export async function getWorkspaceDiagnostics(fileUri: vscode.Uri): Promise {
  // Chờ 1.5 giây để Linter/TypeScript Compiler cập nhật bảng lỗi
  await new Promise(resolve => setTimeout(resolve, 1500));

  const diagnostics = vscode.languages.getDiagnostics(fileUri);
  const errors = diagnostics
    .filter(d => d.severity === vscode.DiagnosticSeverity.Error)
    .map(d => `Line ${d.range.start.line + 1}: ${d.message} [${d.source}]`);

  return errors;
}
```

5. QUẢN LÝ RỦI RO & PHÂN QUYỀN BẢO MẬT (SECURITY FRAMEWORK)

Chính Sách An Toàn Cấp Độ Doanh Nghiệp

Khi sử dụng AWS Bedrock trong môi trường doanh nghiệp, tất cả các dữ liệu mã nguồn không bị sử dụng để huấn luyện model (Zero Data Retention). Extension áp dụng cơ chế phân quyền 3 tầng để ngăn chặn các thao tác độc hại hoặc vô tình phá hoại dự án.

- **Cấp 1 - Tự Động Duyệt (Auto-Approve):** Đọc file, tìm kiếm regex, liệt kê thư mục, truy xuất AST graph.
- **Cấp 2 - Yêu Cầu Duyệt (Prompt Approval):** Sửa file code, tạo file mới, cài đặt thư viện `npm install`.
- **Cấp 3 - Cảnh Báo Nguy Hiểm (Strict Security Lock):** Chạy command Terminal có chứa các câu lệnh nguy hiểm (`rm`, `git push --force`, `drop database`, sửa file `.env` chứa bí mật API keys).

6. KẾT LUẬN

Với bản kế hoạch chi tiết 6 Phase và 12 tuần triển khai này, đội ngũ kỹ sư có thể xây dựng một **VS Code Extension chuẩn Claude Code / Antigravity** hoàn chỉnh. Hệ thống không chỉ tận dụng hạ tầng mạnh mẽ và bảo mật của **AWS Bedrock**, mà còn tối ưu hóa chi phí token, tốc độ phản hồi và sự an toàn khi thao tác trực tiếp trên workspace của lập trình viên.